

```
1  /// import libreria http + alias
2  import 'package:http/http.dart' as http;
3
4  /// import dart convert
5  import 'dart:convert';
6  import 'package:hacker_news_api/models/story_model.dart';
7
8  /// per debugPrint
9  import 'package:flutter/foundation.dart';
10
11  /// classe HackerNewsService
12  /// contenente la logica HTTP/JSON,
13  /// logica applicativa/interfacciamento esterno.
14  /// Questa classe gestisce la logica per interfacciarsi
15  /// con il servizio esterno, come la chiamata HTTP
16  /// per recuperare i dati da Hacker News
17  class HackerNewsService {
18    /// numero di topStories da prendere
19    final int _firstTopStories = 15;
20
21    /// metodo downloadStoryData
22    Future<List<StoryModel>> downloadStoryData() async {
23      /// variabile che evita: app bloccata,
24      /// spinner infinito e UX pessima su rete lenta
25      const timeoutDuration = Duration(seconds: 10);
26
27      /// chiamata http per prendere le topstories
28      final topStoriesResponse = await http
29        .get(
30          Uri.parse(
31            "https://hacker-news.firebaseio.com/v0/topstories.json?print=pretty",
32          ),
33        )
34        .timeout(
35          timeoutDuration);
36
37      /// Controllo status code
38      /// Service fallisce in modo controllato
39      if (topStoriesResponse.statusCode != 200) {
40        throw Exception(
41          "Errore HTTP ${topStoriesResponse.statusCode} su topstories",
42        );
43      }
44
45      /// Parsing JSON topstories
46      /// Alla fine dell'istruzione non metto
47      /// "as List<dynamic>", perché non sono
48      /// sicuro che questo è una List.
49      /// Se il body non è JSON valido, json.decode
50      /// lancia FormatException
51      dynamic decodedTopStories;
52      try {
53        decodedTopStories = json.decode(topStoriesResponse.body);
54      } on FormatException catch (e) {
55        throw Exception("JSON non valido per topstories: $e");
56      }
57
58      /// Controllo che "decodedTopStories" sia una lista.
59      /// Se non lo è (se non è una List), allora considero
```

```
61    /// il payload non valido e fallisco.
62    /// Se è una List -> ok, continuo
63    /// Se è null, Map, String, int, ... -> errore controllato
64    if (decodedTopStories is! List) {
65      throw Exception("Formato non valido per topstories");
66    }
67
68    /// Prendo solo i primi firstTopStories id
69    /// e controllo che la lista contenga int
70    final topStoriesIds = decodedTopStories
71      .whereType<int>()
72      .take(_firstTopStories)
73      .toList();
74
75    /// Chiamate parallele per i dettagli delle stories
76    final futures = topStoriesIds.map((int storyId) async {
77      /// si deve iterare/ciclare per ciascun id/elemento
78      /// e per ciascun id fare un'altra chiamata http
79
80      try {
81        /// altra chiamata http per ciascun id, http.get
82        final response = await http
83          .get(
84            Uri.parse(
85              "https://hacker-news.firebaseio.com/v0/item/$storyId.json?
print=pretty",
86            ),
87          )
88          .timeout(timeoutDuration);
89
90        /// Controllo status code item
91        if (response.statusCode != 200) {
92          debugPrint(
93            "HackerNewsService: item $storyId "
94            "HTTP ${response.statusCode} "
95            "(${response.request?.url})",
96          );
97          return null;
98        }
99
100       /// Parsing JSON item
101       dynamic decodedItem;
102       try {
103         decodedItem = json.decode(response.body);
104       } on FormatException catch (e) {
105         debugPrint(
106           'HackerNewsService: item $storyId '
107           'JSON non valido: $e ',
108         );
109         return null;
110       }
111
112       /// Distinguo il caso in cui "decodedItem" è null
113       if (decodedItem == null) {
114         debugPrint('HackerNewsService: item $storyId restituito null');
115         return null;
116       }
117
118       /// Evita crash quando: HackerNews restituisce null,
119       /// arriva payload non previsto, API cambia
```

```
120     if (decodedItem is! Map<String, dynamic>) {
121         debugPrint(
122             'HackerNewsService: item $storyId '
123             'payload non Map ({decodedItem.runtimeType})',
124         );
125         return null;
126     }
127
128     /// ritorna un'istanza di StoryModel
129     /// invocando il metodo fromData
130     return StoryModel.fromData(decodedItem);
131 } catch (e, s) {
132     debugPrint(
133         'HackerNewsService: item $storyId '
134         'errore inatteso: $e',
135     );
136     debugPrint(s.toString());
137     return null;
138 }
139 }).toList();
140
141 /// Attendo tutte le richieste
142 final results = await Future.wait<StoryModel?>(futures);
143
144 /// Tengo solo quelle valide
145 final stories = results.whereType<StoryModel>().toList();
146
147 // ritorno le stories
148 return stories;
149 }
150 }
151
```